

bipl-lsp

Editing BIPL, properly.

An LSP language server for a language from the Software Languages Book.

```
factorialV1.bipl - softlang/yas

1 while (i <= x) { x : int · inferred program input
2   y = y * i;
3 }
```

The claim: even a tiny language hides two real problems worth tooling.

The assignment — and why BIPL

OPTION LSP.2

Build an LSP editor for a recurring language in the Software Languages Book (YAS).

I chose BIPL

a small imperative language — a fragment of C.

Assignment, if, while, and integer / boolean expressions.

No declarations, no input/output.

SMALL — BUT WORTH TOOLING

BIPL has real static semantics behind its five statements:

- **type consistency** — arithmetic vs. boolean; conditions must be boolean
- **definite assignment** — a variable shouldn't be read before it is assigned

So the diagnostics come from an actual analysis — not from matching keywords.

Approach: reuse the lecture's LSP setup

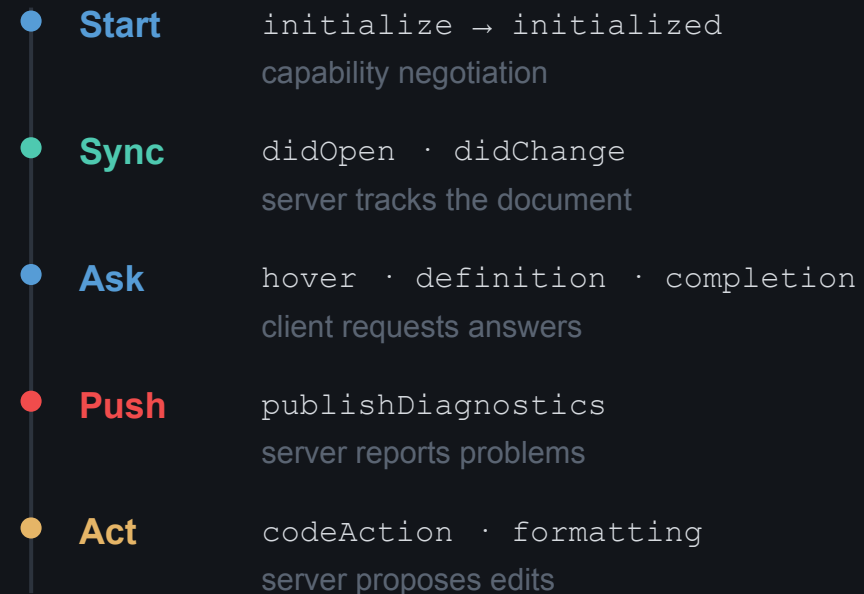
Same stack the LSP lecture demoed for FSML: a `pygls` server + a VS Code client, talking `JSON-RPC` over stdio.

One request on the wire

```
{ "jsonrpc": "2.0", "id": 7,  
  
  "method": "textDocument/hover",  
  
  "params": { "position": {"line":0,"character":12} }  
}
```

“what's at line 0, column 12?” → the server answers from its analysis.

The session lifecycle (from the lecture)



What's mine: I reproduced that architecture and swapped in a much richer analysis for BIPL.

Three facts that drive everything

“A trivial imperative programming language ... a small fragment of C.” — softlang/yas

```
division.bipl
1 // Euclidean division
2 {
3     q = 0;
4     r = x;
5     while (r >= y) {
6         r = r - y;
7         q = q + 1;
8     }
9 }
```

What BIPL leaves implicit

No declarations. You never write `int x`. The editor must infer every type — nobody states it.

No boolean literals. No `true` / `false` keyword. Booleans exist only as results of comparisons and `&&` `||` `!`.

Inputs live in the store. No `input` statement — a program just reads variables already in memory. That is why samples read `x` without assigning it.

How the analyzer works — inside analysis.py

One file, one function: text goes in, diagnostics come out.

```
analysis.py

1 def analyze(text):
2     tokens = tokenize(text)
3     ast     = Parser(tokens).parse_program()
4     chk     = Checker()
5     chk.collect_assigned(ast)
6     chk.check_stmt(ast, set())
7     return AnalysisResult(
8         ast, diagnostics, variables)
```

Four stages, one pass

1 Lexer `tokenize()`

splits the text into words — numbers, names, symbols — and skips spaces and // comments.

2 Parser `recursive descent`

arranges the words into a tree (the AST) following the grammar; recovers from typos.

3 Checker `one walk of the tree`

type-checks and flow-checks it, recording every variable and where it appears.

4 Result `AnalysisResult`

the diagnostics (the squiggles) plus a table every editor feature reads from.

The two checks: types and flow

Type checking

types.bipl

```
1 x = 5;
2 x = 1 < 2;  int can't become bool
```

CHAPTER 9 · WELL-TYPEDNESS

$m \vdash e : T$ in context m , expression e has type T

$$\frac{e0 : \text{bool} \quad e1 : T \quad e2 : T}{\text{if } (e0) \text{ } e1 \text{ else } e2 : T} [\text{t-if}]$$

Our checker implements this — and completes the operators the YAS Haskell left as “TODO”.

Definite assignment

A read is safe only if the variable is assigned on every path. An if needs both branches; a while body may run zero times. (A data-flow pass — beyond Ch. 9.)

flow.bipl

```
1 if (c) y = 1;
2 z = x;  y maybe uninitialized
3
4 while (c) v = 1;
5 u = x;  v maybe uninitialized
```

One result, every feature. Hover, definition, rename, completion and the outline are all just queries on this.

Not every unassigned read is a bug

BIPL reads its inputs from the store. So a naive “read-before-write = error” rule would flag `factorialV1.bipl` — an official sample — as broken. **The language would reject its own textbook.**

PROBLEMS

- ✖ **Error**
`type-error`
conditions must be boolean; a variable can't change type
- ⚠ **Warning**
`maybe-uninitialized`
assigned somewhere, but not on every path to this read
- ℹ **Information**
`input-variable`
never assigned anywhere → a program input; this is fine

12/12

official YAS samples
analyse with zero errors

— and a test locks it in,
so it can never regress.

Telling a genuine bug apart from a legitimate input — that is the real work here.

Demo — in action

bad.bipl

```
1 {  
2   x = 5;  
3   if (x) y = 1;   condition must be boolean  
4   z = y + 1;   y maybe uninitialized  
5   w = (2 < 3) + 4;   '+' expects integers  
6 }
```

PROBLEMS ✖ 3 errors ⚠ 2 warnings

1

good.bipl

blue input hints; hover shows types inferred from use

2

bad.bipl

3 type errors + 2 data-flow warnings, live

3

Navigate

hover · go-to-definition · find references

4

Refactor

F2 rename (rejects invalid names) · quick fix

5

Live typing

break factorial, watch diagnostics move as you type

Challenges & loose ends

What I had to solve — and what I deliberately left open.

✓ Solved

Inputs vs. bugs. A naive rule broke the samples → three severities + an “assigned-anywhere” pre-pass.

Types without declarations. Demand-driven inference: a constrained use ($i \leq x$) pins the type.

Completed the checker. The YAS TypeChecker.hs ended at “TODO”; I implemented the full operator set.

Stayed useful while broken. Error recovery: skip to the next ; or } so half-typed code still analyses.

○ Left open (on purpose)

No full type unification. A variable used only neutrally ($r = x$) stays unknown — I infer from constrained uses only.

Re-analysis per keystroke. Whole-document, not incremental. Linear and instant at BIPL's scale; wouldn't scale to big files.

Fidelity vs. usability. Relaxed the Haskell checker's both-branches-equal rule, and accept the conventional identifier shape (a superset of `Is.term`).

Three things to remember

1

LSP decouples smartness from the editor.

One Python analysis serves any LSP editor; the VS Code client is 30 lines.

2

A small language is not trivial tooling.

Five statements still forced type inference without declarations and branch-sensitive data flow.

3

The language's own samples were the spec.

They told me inputs aren't errors — and now they guard that decision as tests. This is the FSML lecture demo, extended to a harder language.